VHDL 과제 1 보고서

디지털 논리회로 설계 · VHDL_1 문제 1–5

문제 1. out1 <= a and b and c and d, out2 <= a or b or c or d

설명

입력 a, b, c, d 네 개와 출력 out1, out2 두 개를 가진 조합회로입니다. 두 개의 동시 신호 할당문으로 4입력 AND(out1)와 4입력 OR(out2)를 구현했습니다.

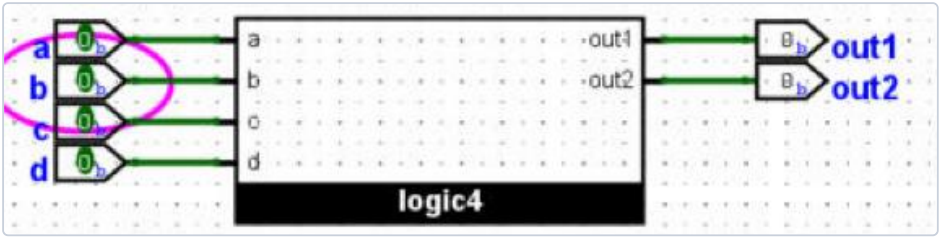
VHDL 코드

```vhdl
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

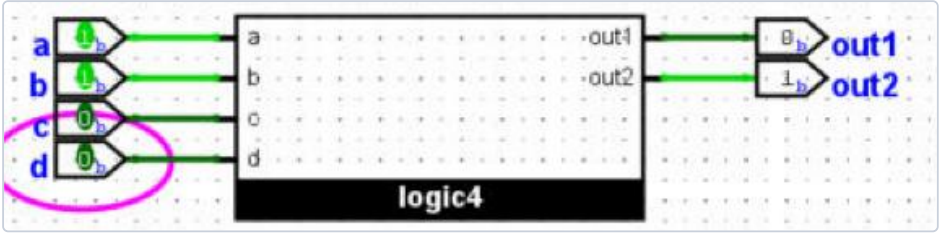
--entity
entity logic4 is
     port(
          a,b,c,d    :in  std_logic;
          out1,out2  :out std_logic
     );
end entity logic4;

--architecture
architecture bhv of logic4 is
begin
     out1 <= a and b and c and d;    -- 4-input AND
     out2 <= a or b or c or d;       -- 4-input OR
end architecture bhv;
```

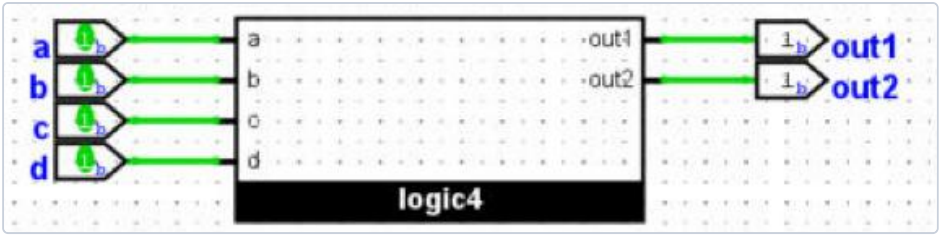
회로 및 시뮬레이션



a b c d = 0000 → out1 = 0, out2 = 0



a b c d = 1100 → out1 = 0, out2 = 1



a b c d = 1111 → out1 = 1, out2 = 1

그림 1. P1_and_or 회로 시뮬레이션

> **결과 설명.** 입력이 모두 1일 때만 out1 = 1이 되고, 입력 중 하나라도 1이면 out2 = 1이 됩니다. 모두 0이면 두 출력 모두 0입니다. AND와 OR의 정의대로 동작합니다.

문제 2. 8-to-1 멀티플렉서

설명

데이터 입력 d0–d7과 3비트 선택 신호 `ch(2 downto 0)`를 받아, ch 값에 해당하는 입력 하나를 `data`로 내보냅니다. 강의의 4-to-1 MUX 예제처럼 process 안에서 `case` 문을 사용했습니다.

ch	000	001	010	011	100	101	110	111
data	d0	d1	d2	d3	d4	d5	d6	d7

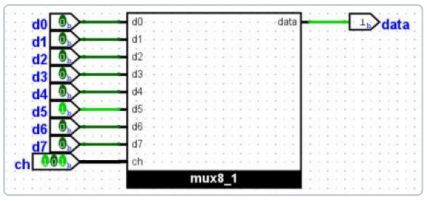
VHDL 코드

```vhdl
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

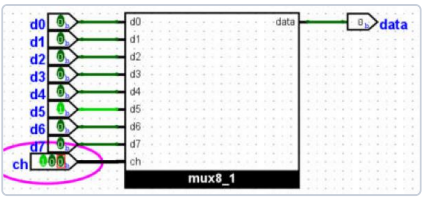
--entity
entity mux8_1 is
     port(
          d0,d1,d2,d3,d4,d5,d6,d7 :in  std_logic;
          ch                      :in  std_logic_vector(2 downto 0);
          data                    :out std_logic
     );
end entity mux8_1;

--architecture
architecture bhv of mux8_1 is
begin
     process(d0,d1,d2,d3,d4,d5,d6,d7,ch)
     begin
          case ch is
               when "000"  => data <= d0;
               when "001"  => data <= d1;
               when "010"  => data <= d2;
               when "011"  => data <= d3;
               when "100"  => data <= d4;
               when "101"  => data <= d5;
               when "110"  => data <= d6;
               when "111"  => data <= d7;
               when others => data <= '0';
          end case;
     end process;
end architecture bhv;
```

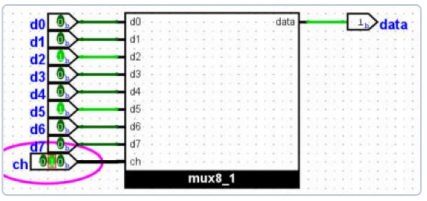
회로 및 시뮬레이션



d5 = 1, ch = 101 → data = 1



d5 = 1, ch = 100 → data = 0



d2 = 1, ch = 010 → data = 1

그림 2. P2_mux8_1 회로 시뮬레이션

> **결과 설명.** d5만 1일 때 ch = 101이면 data = 1, ch = 100(d4 선택)이면 data = 0입니다. d2를 1로 바꾸고 ch = 010으로 선택하면 data = 1이 됩니다. 선택 신호가 가리키는 입력만 출력으로 전달됩니다.

문제 3. 7세그먼트로 글자 a, b, c, d, f, g 표시

설명

강의의 `led` 엔티티와 같은 구조로, 인에이블 `en`, 3비트 선택 신호 `sel`, 8비트 출력 `display(7 downto 0) = a b c d e f g dp`를 사용했습니다. `en = '0'`이면 출력은 고임피던스("ZZZZZZZZ")입니다. 7세그먼트로는 모든 소문자를 그릴 수 없어서 일반적으로 쓰는 모양인 **A, b, C, d, F, g**로 표시했습니다. Logisim에서는 8비트 출력을 스플리터로 나눠 7세그먼트 표시기의 각 핀(a–g, dp)에 연결했습니다.

sel	표시	a	b	c	d	e	f	g	dp
000	**A**	1	1	1	0	1	1	1	0
001	**b**	0	0	1	1	1	1	1	0
010	**C**	1	0	0	1	1	1	0	0
011	**d**	0	1	1	1	1	0	1	0
100	**F**	1	0	0	0	1	1	1	0
101	**g**	1	1	1	1	0	1	1	0
110, 111	**(꺼짐)**	0	0	0	0	0	0	0	0

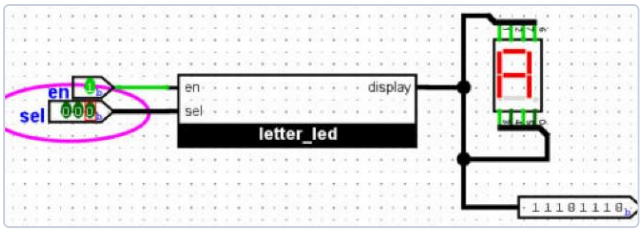
VHDL 코드

```vhdl
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

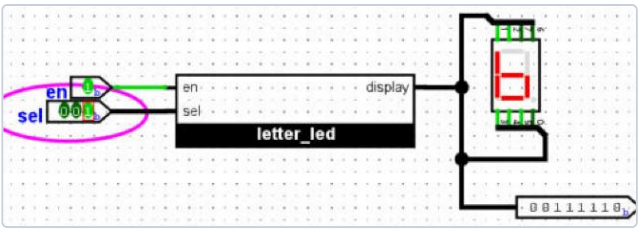
--entity
-- display(7 downto 0) = a b c d e f g dp
entity letter_led is
     port(
          en      :in  std_logic;
          sel     :in  std_logic_vector(2 downto 0);
          display :out std_logic_vector(7 downto 0)
     );
end entity letter_led;

--architecture
architecture bhv of letter_led is
begin
     process(en,sel)
     begin
          if en='0' then
               display <= "ZZZZZZZZ";
          else
               case sel is
                    when "000"  => display <= "11101110";  -- A
                    when "001"  => display <= "00111110";  -- b
                    when "010"  => display <= "10011100";  -- C
                    when "011"  => display <= "01111010";  -- d
                    when "100"  => display <= "10001110";  -- F
                    when "101"  => display <= "11110110";  -- g
                    when others => display <= "00000000";  -- off
               end case;
          end if;
     end process;
end architecture bhv;
```

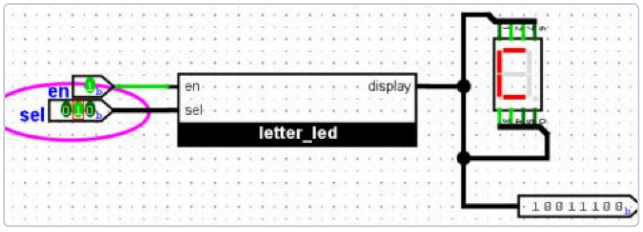
회로 및 시뮬레이션



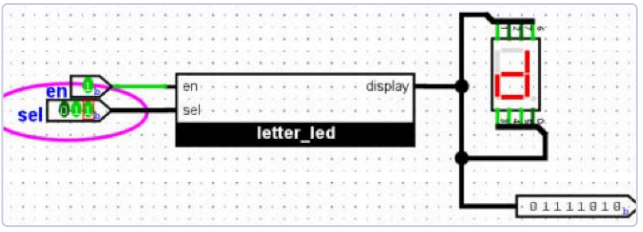
sel = 000 → A



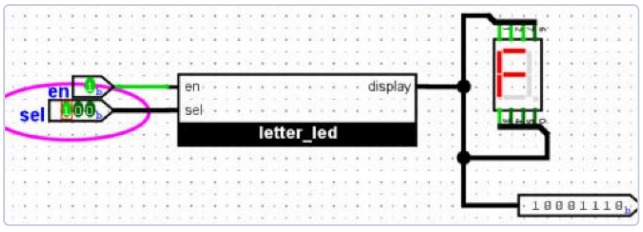
sel = 001 → b



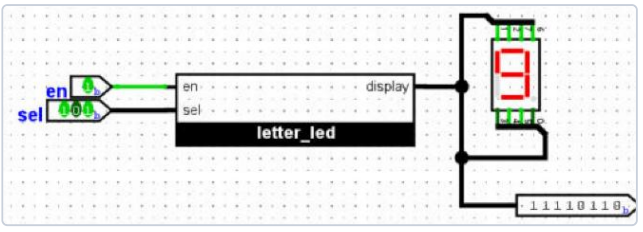
sel = 010 → C



sel = 011 → d



sel = 100 → F



sel = 101 → g

그림 3. P3_letter 회로 시뮬레이션 (en = 1)

> **결과 설명.** sel을 000부터 101까지 바꾸면 A, b, C, d, F, g가 차례로 표시됩니다. 오른쪽 아래 `display` 핀의 8비트 값도 위 표의 세그먼트 코드와 같습니다.

문제 4. 25인 투표기

설명

강의의 13인 투표기를 25명으로 확장했습니다. 입력은 `vt(25 downto 1)` 이고, 찬성(1)이 과반수, 즉 **13표 이상**이면 `result = '1'` 입니다. process 안에서 `for` 문으로 1의 개수를 변수 `sum` 에 세고, 반복문이 끝난 뒤 `sum >= 13` 을 한 번 비교해 결과를 정합니다.

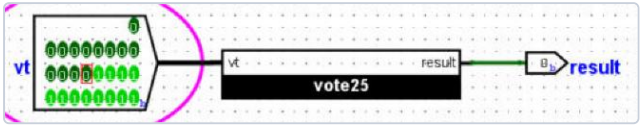
VHDL 코드

```vhdl
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

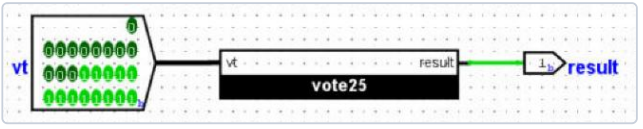
--entity
entity vote25 is
     port(
          vt     :in  std_logic_vector(25 downto 1);
          result :out std_logic
     );
end entity vote25;

--architecture
architecture bhv of vote25 is
begin
     process(vt)
     variable sum:integer range 0 to 25;
     begin
          sum := 0;
          for i in 1 to 25 loop
               if (vt(i)='1') then
                    sum := sum+1;
               end if;
          end loop;
          if sum >= 13 then      -- more than half of 25
               result <= '1';
          else
               result <= '0';
          end if;
     end process;
end architecture bhv;
```

회로 및 시뮬레이션



찬성 12표 → result = 0



찬성 13표 → result = 1

그림 4. P4_vote25 회로 시뮬레이션 (과반 경계: 12표와 13표)

> **결과 설명.** 찬성이 12표(과반 미만)이면 result = 0, 13표(과반)가 되면 result = 1이 됩니다. 25명의 과반 기준인 13표에서 정확히 결과가 바뀝니다.

문제 5. vt의 '1' 개수를 7세그먼트에 표시

설명

13비트 입력 `vt(13 downto 1)`에서 '1'의 개수를 `for` 문으로 세어 변수 `sum`에 저장하고, `case` 문으로 7세그먼트 코드로 바꿉니다. 개수가 0–9이면 숫자, 10–15이면 A, b, C, d, E, F를 표시합니다 (B와 D는 8, 0과 구분되도록 소문자 b, d 모양을 사용). `en = '0'`이면 `"00000000"`을 출력해 표시기를 끄고, `en = '1'`이면 결과를 표시합니다.
13비트이므로 실제 최대값은 13(d)이지만 요구사항에 맞춰 14(E), 15(F)도 포함했습니다.

VHDL 코드

```
--library and package
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

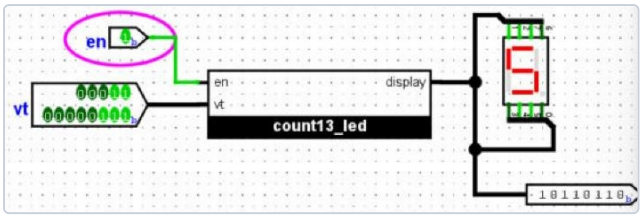
--entity
-- display(7 downto 0) = a b c d e f g dp
entity count13_led is
     port(
          en      :in  std_logic;
          vt      :in  std_logic_vector(13 downto 1);
          display :out std_logic_vector(7 downto 0)
     );
end entity count13_led;

--architecture
architecture bhv of count13_led is
begin
     process(en,vt)
     variable sum:integer range 0 to 15;
     begin
          sum := 0;
          for i in 1 to 13 loop
               if (vt(i)='1') then
                    sum := sum+1;
               end if;
          end loop;
          if en='0' then
               display <= "00000000";               -- display off
          else
               case sum is
                    when 0      => display <= "11111100";  -- 0
                    when 1      => display <= "01100000";  -- 1
                    when 2      => display <= "11011010";  -- 2
                    when 3      => display <= "11110010";  -- 3
                    when 4      => display <= "01100110";  -- 4
                    when 5      => display <= "10110110";  -- 5
                    when 6      => display <= "10111110";  -- 6
                    when 7      => display <= "11100000";  -- 7
                    when 8      => display <= "11111110";  -- 8
                    when 9      => display <= "11110110";  -- 9
                    when 10     => display <= "11101110";  -- A
                    when 11     => display <= "00111110";  -- b
                    when 12     => display <= "10011100";  -- C
                    when 13     => display <= "01111010";  -- d
                    when 14     => display <= "10011110";  -- E
                    when 15     => display <= "10001110";  -- F
                    when others => display <= "00000000";
               end case;
          end if;
     end process;
end architecture bhv;
```

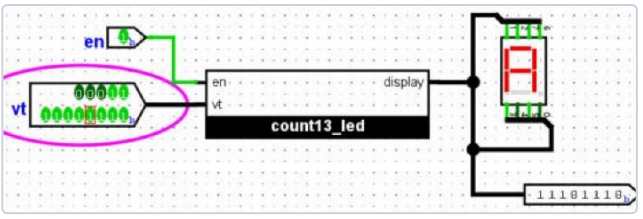
개수	표시	a	b	c	d	e	f	g	dp
0	**0**	1	1	1	1	1	1	0	0
1	**1**	0	1	1	0	0	0	0	0
2	**2**	1	1	0	1	1	0	1	0
3	**3**	1	1	1	1	0	0	1	0
4	**4**	0	1	1	0	0	1	1	0
5	**5**	1	0	1	1	0	1	1	0
6	**6**	1	0	1	1	1	1	1	0
7	**7**	1	1	1	0	0	0	0	0

개수	표시	a	b	c	d	e	f	g	dp
8	**8**	1	1	1	1	1	1	1	0
9	**9**	1	1	1	1	0	1	1	0
10	**A**	1	1	1	0	1	1	1	0
11	**b**	0	0	1	1	1	1	1	0
12	**C**	1	0	0	1	1	1	0	0
13	**d**	0	1	1	1	1	0	1	0
14	**E**	1	0	0	1	1	1	1	0
15	**F**	1	0	0	0	1	1	1	0

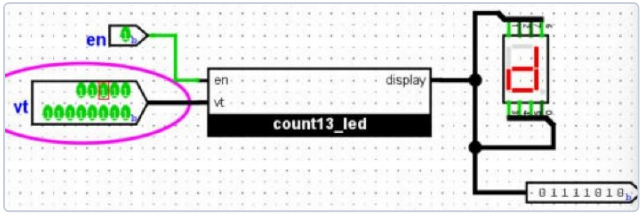
회로 및 시뮬레이션



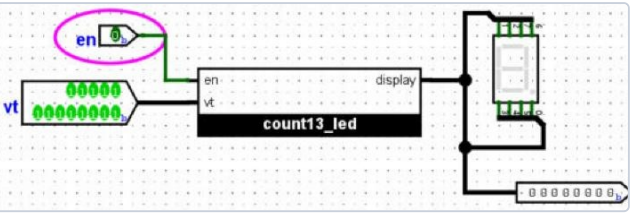
en = 1, '1' 5개 → 5



en = 1, '1' 10개 → A



en = 1, '1' 13개 → d



en = 0 ('1' 13개) → 꺼짐

그림 5. P5_count13 회로 시뮬레이션

> **결과 설명.** en = 1일 때 '1'이 5개면 5, 10개면 A, 13개면 d가 표시되어 개수에 맞는 숫자·문자가 나옵니다. 같은 입력에서 en = 0으로 바꾸면 모든 세그먼트가 꺼집니다(display = 00000000). 요구사항대로 동작합니다.